HW3: Model Selection (due 11/22 9:00)

Name: 莊明儒 Student ID: 111024513

Contents

0.1 Problem 1: Chemical Manufacturing Process Data 1

0.2 Problem 2: Data with Outliers 23

0.3 Appendix 26

0.0.1 Data Source

```
library(AppliedPredictiveModeling) #install package first!!
library(corrplot)  #correlation plot
library(GGally)
library(leaps)
library(glmnet)
library(kableExtra)
library(plotrix)
```

0.1 Problem 1: Chemical Manufacturing Process Data

This data set consists of 177 samples of biological material for which 58 characteristics were measured. Of the 58 characteristics, there were 12 measurements of the biological starting material; 45 measurements of the manufacturing process, and one variable of the resulting final product yield (target variable).
The objective of this problem is to predict the yield given the other variable information.

More data descriptions can be found by typing `help(ChemicalManufacturingProcess)` in r command.

0.1.1 View data

Add your own EDA…

```
data(ChemicalManufacturingProcess)
#View(ChemicalManufacturingProcess)  #you may view data in table
#help(ChemicalManufacturingProcess)
#view(dfSummary(ChemicalManufacturingProcess))  #view data summary in html
```

0.1.2 Data preprocessing

- There are several NA's in the data. We will only use the data with the complete variable information in this homework analysis.
- Rename the dataframe consisting of the complete variable information as **CMP**.
- Rename the variable for a shorten expression

```
dim(ChemicalManufacturingProcess)
```

```
## [1] 176  58
```

```
sum(is.na(ChemicalManufacturingProcess))
```

```
## [1] 106
```

```
which(apply(is.na(ChemicalManufacturingProcess),1,sum)>0) #check which data sample has missing
```

```
##   1   2   3   4   5   6  15  16  17  18  19  20  22  23  24  90  98 134 139 172
##   1   2   3   4   5   6  15  16  17  18  19  20  22  23  24  90  98 134 139 172
## 173 174 175 176
## 173 174 175 176
```

```
CMP <- na.omit(ChemicalManufacturingProcess) #remove missing data
dim(CMP)
```

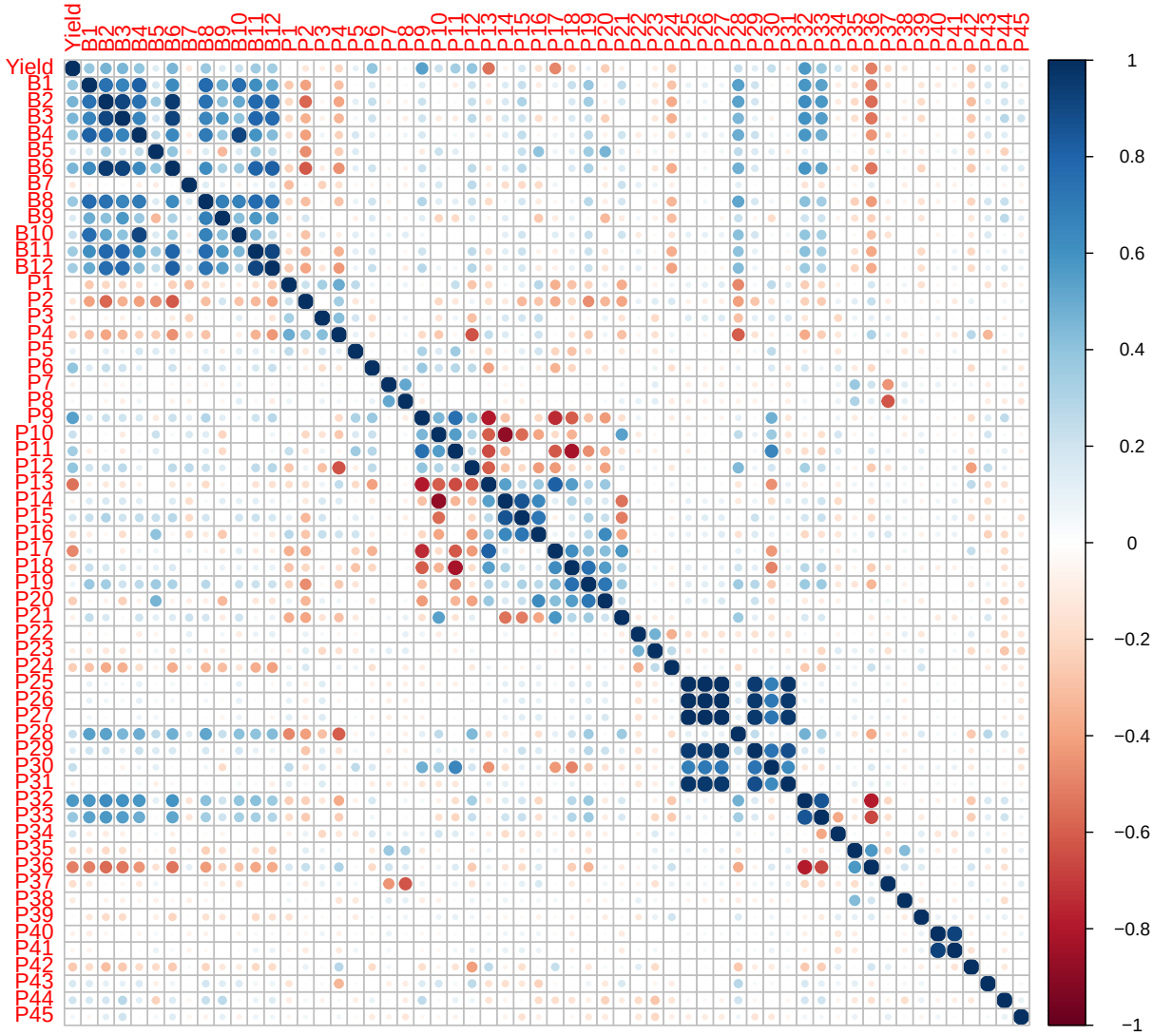
```
## [1] 152  58
```

```
sum(is.na(CMP))
```

```
## [1] 0
```

```
#rename variable
B_name = c()
for (i in 1:12){
  B_name[i] = paste("B",i,sep="")
}
P_name = c()
for (i in 1:45){
  P_name[i] = paste("P",i,sep="")
}
names(CMP) <- c("Yield",B_name, P_name)

corrplot(cor(CMP))
```



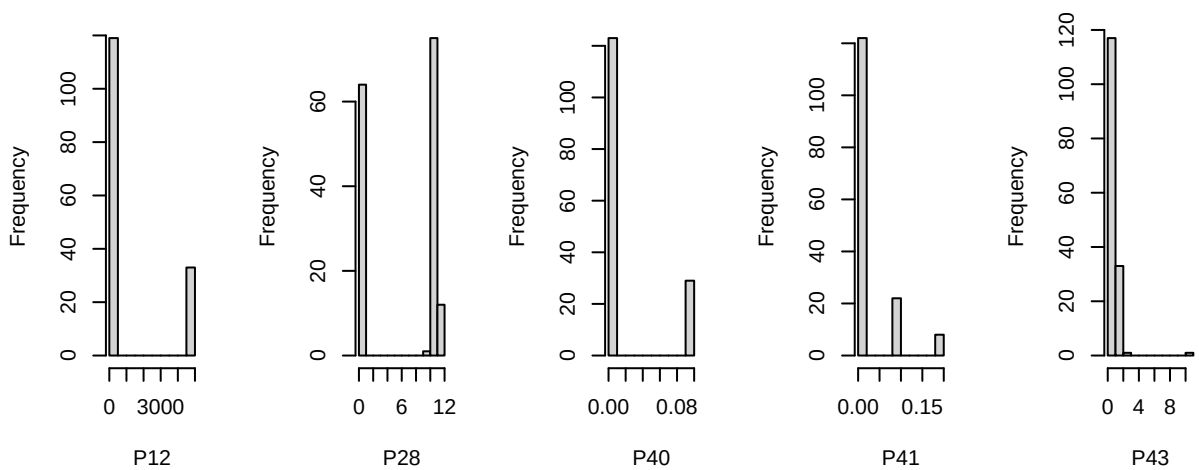
0.1.3 Your analysis should include the following:

- Fit regression models with various variable selection procedures.
- Go through all variable selection procedures taught in the class, to determine your best model (possibly more than one).
- Identify the important input variables (or any new features you defined) and carefully describe their impacts on the response *yield*.
- Evaluate your prediction performance via 5-fold cross validation.
- Give a brief analysis summary.

0.1.4 EDA

Since the number of variables is quite huge in this data set, it seems that only basic summary table can be shown for each variable at this stage. Due to the space consideration, the summary tables are shown in Appendix A. The tables display five-number-summary and means. From those tables, one can see that most of the variables (except for those such as P12, P28, P40, P41 and P43) might have symmetric distributions since their means are very similar to their medians. The empirical distribution of P12, P28, P40, P41 and P43 are shown as below.

```
par(mfrow = c(1, 5))
hist(CMP$P12, xlab = "P12", main = "")
hist(CMP$P28, xlab = "P28", main = "")
hist(CMP$P40, xlab = "P40", main = "")
hist(CMP$P41, xlab = "P41", main = "")
hist(CMP$P43, xlab = "P43", main = "")
```



Furthermore, there are also some predictors that have several outliers such as P25~P31. Such data might have high leverage and could become influential points during regression analysis. Related issue should be checked if model diagnostics are also required. Other points that might be worth noticing are listed as follows:

1. According to the correlation plot shown above, there are several predictors that are very similar to each other, implying that they might share very similar information in predicting Yield. Hence, it should be possible to reduce the dimensionality of the data through variable selection or some useful projection with out losing too much valuable information.

2. According to the first column of the correlation plot, there does not seem to be so many predictors that are highly associated with Yield. Such fact might be consistent with sparsity principle, and hence provides reasonable justification to conduct variable selection.

3. From the summary tables in Appendix A, it is shown that the predictors differ quite a lot in their scale. Hence, standardization of the predictors seems to be necessary to conduct in first if the magnitudes of the coefficients for the predictors are going to be used to judge the importance of them.

0.1.5 Fit regression models with various variable selection procedures.

Since there are too many predictors, it seems necessary to conduct variable selection (data reduction) prior to any further meaningful analysis. Furthermore, to evaluate the prediction performance of each fitted model during the analysis, the dataset is splittet into training set and testing set. The training set and the testing set respectively accounts for 90% (137 samples) and 10% (15 observations) of the original data. The splitting is determined by simple random sampling without replacement (with random seed = 10).

```
train = rep(FALSE, nrow(CMP))
set.seed(10)
train[sample(1:nrow(CMP), round(nrow(CMP) * 0.9))] = TRUE
test = !train
```

Six kinds of variable selection procedures will be performed in the following, including forward selection, backward selection best subset selection, PCR (principal component regression), Lasso regression and Ridge regression. For each procedure, 5-fold cross-validation is used to evaluate the test MSE (mean squared error). For more details, cross validation is used to select "the number of predictors" for the first four procedures and select the tuning parameter λ for Lasso and Ridge regression. The five folds used for cross-validation is randomly determined as follows.

```
k = 5
set.seed (10)
folds=sample(1:k, nrow(CMP), replace=TRUE)
cv.errors=matrix(NA, k, 25)
```

Moreover, since there are only five folds without a lot samples in each of them, it is believed that using "median", "minimum" and "maximum" should be a more appropriate way to summarize the test MSEs under each setting.

a. Forward selection

```
for(j in 1:k){
  for.fit = regsubsets(Yield~., data = CMP, subset = train & folds != j,
                       method = "forward", nvmax = 25)
  for(i in 1:25){
    pred = predict.regsubsets(for.fit, CMP[train & folds == j,], id = i)
    cv.errors[j, i] = mean((CMP[train & folds == j, 1]-pred)^2)
  }
}
matplot(cbind(apply(cv.errors, 2, median),
        apply(cv.errors, 2, max) ,
        apply(cv.errors, 2, min)), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "number of predictors", ylab = "test MSE")
abline(v = which.min(apply(cv.errors, 2, median)), col = "red")
legend("topleft", c("median", "min & max"), lty = 1:2)
```

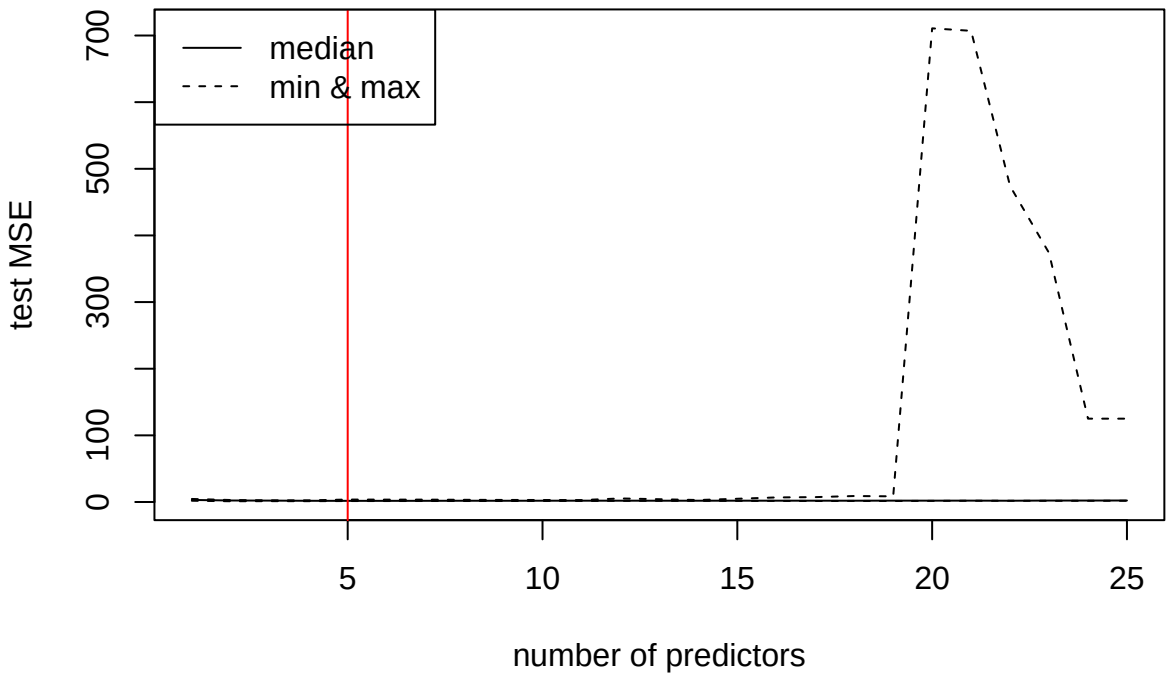


Figure 1: Cross validation with forward selection

Figure 1 shows how the test MSE would vary as increasing the number of predictors under forward selection procedure. The real line and the two dashed lines respectively refer to the median, minimum and maximum of the test MSEs under each number of predictors. According to the median of the test MSEs (i.e., the real line), it can be shown that 5 predictors (indicated by the red line) might provide the lowest test MSE.

```
for.fit = regsubsets(Yield~., data = CMP, subset = train,
                     method = "forward", nvmax = 5)
```

Identify the important input variables

```
coef.names = names(coef(for.fit, id = 5))[-1]
coef.names
```

```
## [1] "P2"  "P9"  "P17" "P33" "P34"
```

According to the result of forward selection, it follows that P2, P9, P17, P33 and P34 are identified as the most important predictors. Hence, the following model is fitted using the whole training data.

$$Yield = \beta_0 + \beta_1 P2 + \beta_2 P9 + \beta_3 P17 + \beta_4 P33 + \beta_5 P34 + \epsilon \tag{1}$$

```r
fit1 = lm(Yield~P2+P9+P17+P33+P34, data = CMP, subset = train)
summ.kable(summary(fit1)$coef, "Summary for fitted model 1")
```

Table 1: Summary for fitted model 1

	Estimate	SE	t-value	p-value
(Intercept)	-5.0561080	12.6941068	-0.3983036	0.6910548
P2	-0.0503211	0.0184120	-2.7330557	0.0071422
P9	0.2850550	0.1155624	2.4666765	0.0149288
P17	-0.6262120	0.1517150	-4.1275540	0.0000648
P33	0.4024508	0.0501702	8.0217167	0.0000000
P34	11.6727722	2.1625781	5.3976188	0.0000003

Describe the impacts of the predictors on Yield

Table 1 shows the fitted result of Model (1), one can see that all the predictors included in the model are significant at 0.05 significance level. According to the sign of the coefficients, it might be able to conclude that after adjusting for the other predictors, P2 and P17 might be negatively associated with Yield, and P9, P33 and P34 might be positively associated with Yield.

b. Backward selection

```r
for(j in 1:k){
  back.fit = regsubsets(Yield~., data = CMP, subset = train & folds != j,
                        method = "backward", nvmax = 25)
  for(i in 1:25){
    pred = predict.regsubsets(back.fit, CMP[train & folds == j,], id = i)
    cv.errors[j, i] = mean((CMP[train & folds == j, 1]-pred)^2)
  }
}
matplot(cbind(apply(cv.errors, 2, median) ,
        apply(cv.errors, 2, max) ,
        apply(cv.errors, 2, min)), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "number of predictors", ylab = "test MSE")
abline(v = which.min(apply(cv.errors, 2, median)), col = "red")
legend("topleft", c("median", "min & max"), lty = 1:2)
```

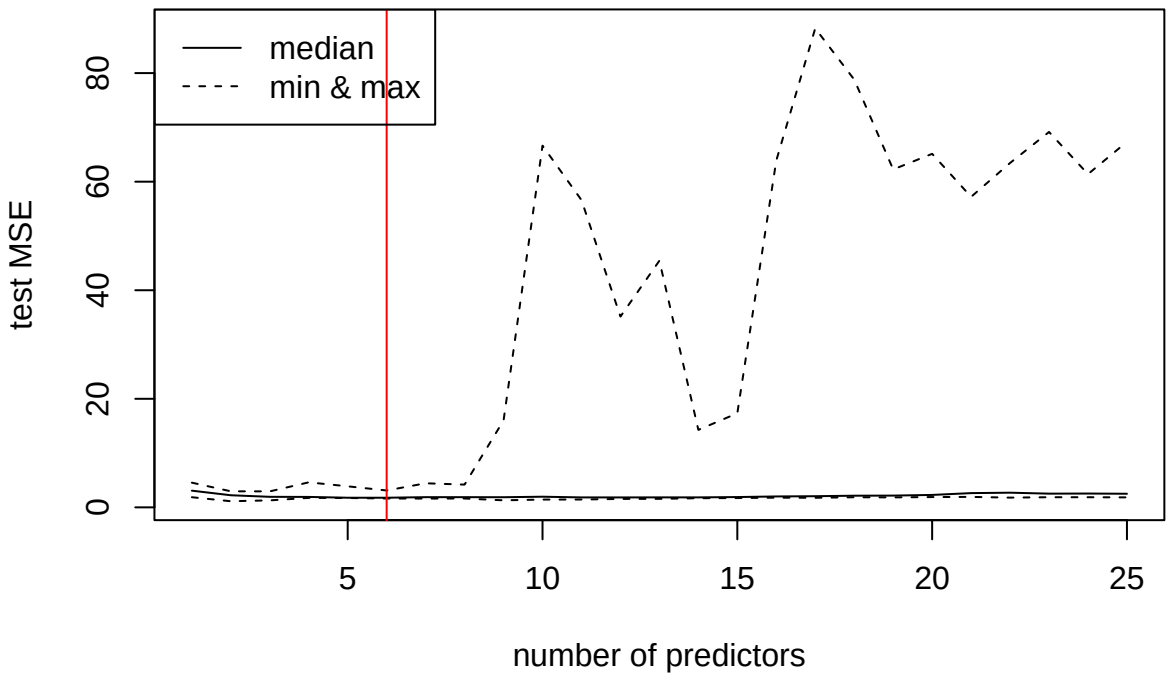


Figure 2: Cross validation with backward selection

Figure 2 shows how the test MSE would vary as increasing the number of predictors under backward selection procedure. The real line and the two dashed lines respectively refer to the median, minimum and maximum of the test MSEs under each number of predictors. According to the median of the test MSEs (i.e., the real line), it can be shown that 3 predictors (indicated by the red line) might provide the lowest test MSE.

```
back.fit = regsubsets(Yield~., data = CMP, subset = train,
                      method = "backward", nvmax = 3)
```

Identify the important input variables

```
coef.names = names(coef(back.fit, id = 3))[-1]
coef.names
```

```
## [1] "P13" "P15" "P33"
```

According to the result of backward selection, it follows that P13, P15 and P33 are identified as the most important predictors. Hence, the following model is fitted using the whole training data.

$$Yield = \beta_0 + \beta_1 P13 + \beta_2 P15 + \beta_3 P33 + \epsilon \tag{2}$$

```r
fit2 = lm(Yield~P13+P15+P33, data = CMP, subset = train)
summ.kable(summary(fit2)$coef, "Summary for fitted model 2")
```

Table 2: Summary for fitted model 2

	Estimate	SE	t-value	p-value
(Intercept)	-1.4685446	13.5436474	-0.1084305	0.9138176
P13	-1.1661746	0.1148632	-10.1527295	0.0000000
P15	0.0109606	0.0023688	4.6270517	0.0000087
P33	0.2487628	0.0499068	4.9845489	0.0000019

Describe the impacts of the predictors on Yield

Table **??** shows the fitted result of Model (2), one can see that all the predictors included in the model are significant at 0.05 significance level. According to the sign of the coefficients, it might be able to conclude that after adjusting for the other predictors, P13 might be negatively associated with Yield, and P15 and P33 might be positively associated with Yield.

c. Best subset selection

Considering the huge computation demand for best subset selection and the fact that both forward and back ward selection favored small number of predictors (5 and 3), the maximum number of predictors is only set up to 6.

```r
cv.errors=matrix(NA, k, 6)
for(j in 1:k){
  best.fit = regsubsets(Yield~., data = CMP, subset = train & folds != j,
                        nvmax = 6, really.big = TRUE)
  for(i in 1:6){
    pred = predict.regsubsets(best.fit, CMP[train & folds == j,], id = i)
    cv.errors[j, i] = mean((CMP[train & folds == j, 1]-pred)^2)
  }
}
matplot(cbind(apply(cv.errors, 2, median) ,
        apply(cv.errors, 2, max) ,
        apply(cv.errors, 2, min)), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "number of predictors", ylab = "test MSE")
abline(v = which.min(apply(cv.errors, 2, median)), col = "red")
legend("topleft", c("median", "min & max"), lty = 1:2)
```

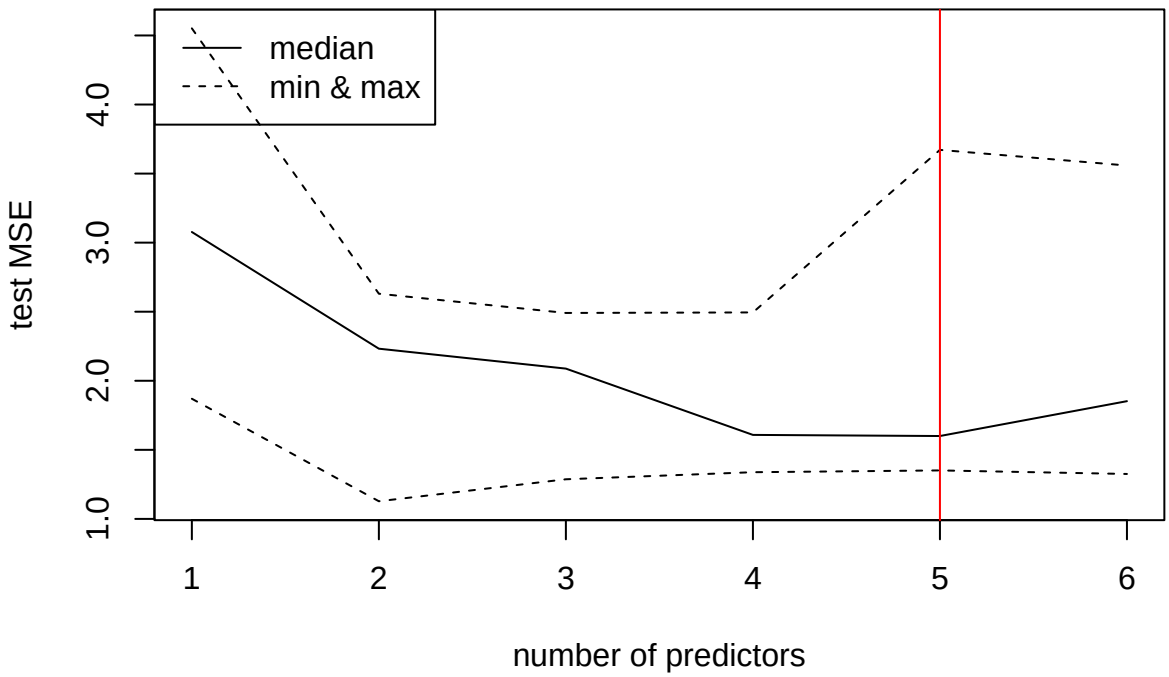


Figure 3: Cross validation with best subset selection

Figure 3 shows how the test MSE would vary as increasing the number of predictors under best subset selection procedure. The real line and the two dashed lines respectively refer to the median, minimum and maximum of the test MSEs under each number of predictors. According to the median of the test MSEs (i.e., the real line), it can be shown that 5 predictors (indicated by the red line) might provide the lowest test MSE.

```
best.fit = regsubsets(Yield~., data = CMP, subset = train,
                      method = "backward", nvmax = 5)
```

Identify the important input variables

```
coef.names = names(coef(best.fit, id = 5))[-1]
coef.names
```

```
## [1] "B3"  "P13" "P15" "P33" "P34"
```

According to the result of forward selection, it follows that B3, P13, P15, P33 and P34 are identified as the most important predictors. Hence, the following model is fitted using the whole training data.

$$Yield = \beta_0 + \beta_1 B3 + \beta_2 P13 + \beta_3 P15 + \beta_4 P33 + \beta_5 P34 + \epsilon \tag{3}$$

```
fit3 = lm(Yield~B3+P13+P15+P33+P34, data = CMP, subset = train)
summ.kable(summary(fit3)$coef, "Summary for fitted model 3")
```

Table 3: Summary for fitted model 3

	Estimate	SE	t-value	p-value
(Intercept)	-31.9630862	14.2161376	-2.248366	0.0262235
B3	0.0978543	0.0303850	3.220477	0.0016141
P13	-1.0600327	0.1034495	-10.246860	0.0000000
P15	0.0103881	0.0021259	4.886471	0.0000029
P33	0.2379853	0.0569701	4.177373	0.0000535
P34	9.7563481	2.1343336	4.571145	0.0000111

Describe the impacts of the predictors on Yield

Table **??** shows the fitted result of Model (3), one can see that all the predictors included in the model are significant at 0.05 significance level. According to the sign of the coefficients, it might be able to conclude that after adjusting for the other predictors, P13 might be negatively associated with Yield, and P3, P15, P33 and P34 might be positively associated with Yield.

d. Principal component regression

According to EDA, we found that there is large difference in scale among the predictors. To avoid the possibility that some of the predictors might dominate the others merely due to their large scales, all the predictors were first standardized before undergoing principal component analysis.

```
X = scale(CMP[,-1])
PC = eigen(t(X) %*% X)$vectors
lambda = eigen(t(X) %*% X)$values
CMP2 = as.data.frame(cbind(CMP$Yield, X %*% PC))
colnames(CMP2) = c("Yield", paste("PC", 1:57, sep = ""))
plot(cumsum(lambda)/sum(lambda), type = "l", xlab = "number of PC",
     ylab = "proportion of total variation")
```

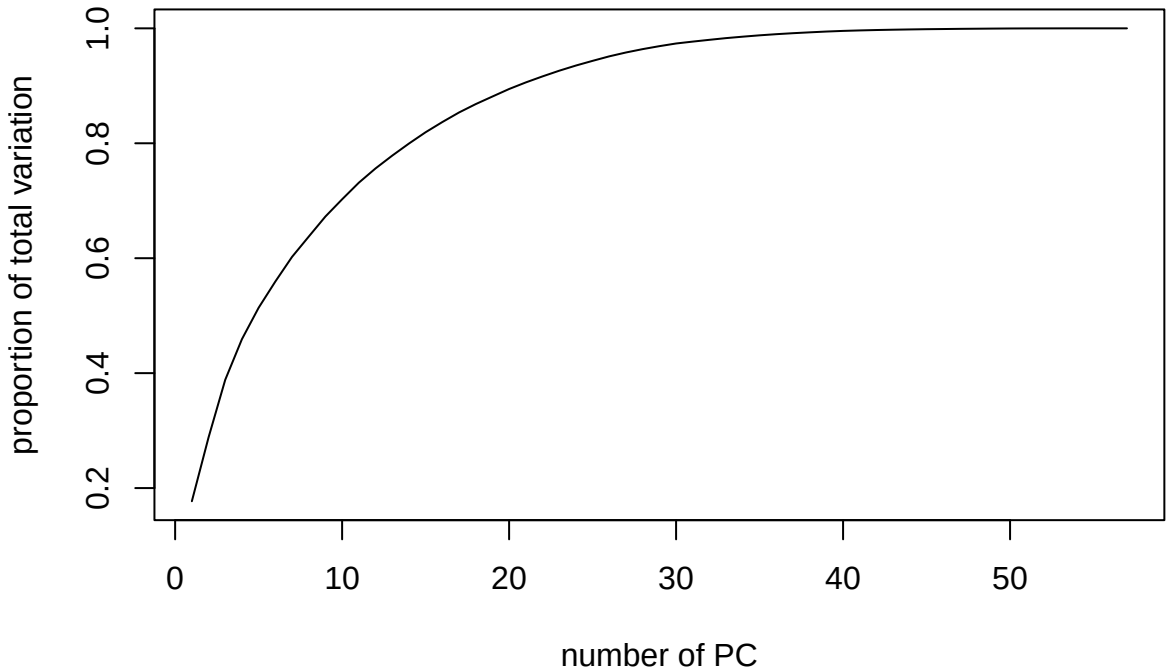


Figure 4: Proportion of total variation of the predictors

Figure 4 shows the cumulated proportion of total variation accounted by the PCs. Notice that the cumulated variation is calculated by summing together the PCs in order according to the respective variation of each of them from large to small.

```
cv.errors=matrix(NA, k, 25)
for(j in 1:k){
  for.fit = regsubsets(Yield~., data = CMP2, subset = train & folds != j,
                       method = "forward", nvmax = 25)
  for(i in 1:25){
    pred = predict.regsubsets(for.fit, CMP2[train & folds == j,], id = i)
    cv.errors[j, i] = mean((CMP2[train & folds == j, 1]-pred)^2)
  }
}
matplot(cbind(apply(cv.errors, 2, median) ,
        apply(cv.errors, 2, max) ,
        apply(cv.errors, 2, min)), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "number of PC predictors", ylab = "test MSE")
abline(v = which.min(apply(cv.errors, 2, median)), col = "red")
legend("topleft", c("median", "min & max"), lty = 1:2)
```

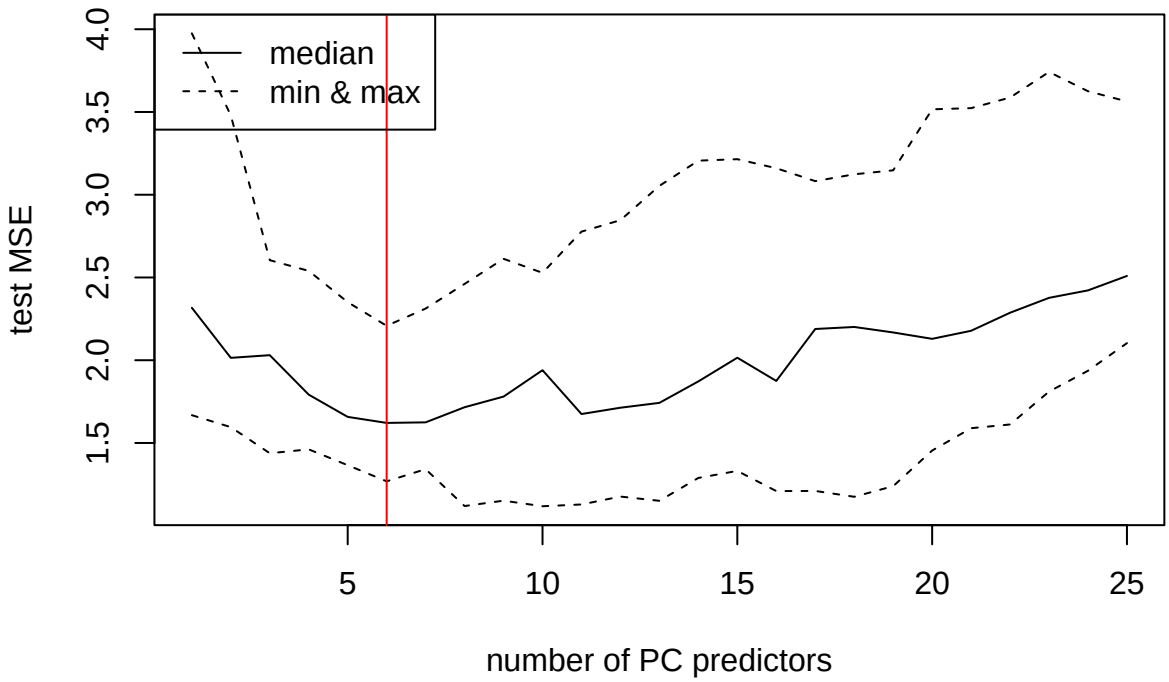


Figure 5: Cross validation with PCR

Figure 5 shows how the test MSE would vary as increasing the number of PC predictors under forward selection procedure. The reason why best subset selection is not adopted here is only due to the consideration of computation convenience. The real line and the two dashed lines respectively refer to the median, minimum and maximum of the test MSEs under each number of predictors. According to the median of the test MSEs (i.e., the real line), it can be shown that 6 PC predictors (indicated by the red line) might provide the lowest test MSE..

```
best.fit = regsubsets(Yield~., data = CMP2, subset = train,
                      method = "backward", nvmax = 6)
```

Identify the important input variables

```
coef.names = names(coef(best.fit, id = 6))[-1]
coef.names
```

```
## [1] "PC1"  "PC2"  "PC4"  "PC8"  "PC9"  "PC26"
```

According to the result of forward selection, it follows that PC1, PC2, PC4, PC8, PC9 and PC26 are identified as the most important predictors. Hence, the following model is fitted using the whole training data.

$$Yield = \beta_0 + \beta_1 PC1 + \beta_2 PC2 + \beta_3 PC4 + \beta_4 PC8 + \beta_5 PC9 + \beta_6 PC26 + \epsilon \tag{4}$$

```
fit4 = lm(Yield~PC1+PC2+PC4+PC8+PC9+PC26, data = CMP2, subset = train)
summ.kable(summary(fit4)$coef, "Summary for fitted model 4")
```

Table 4: Summary for fitted model 4

	Estimate	SE	t-value	p-value
(Intercept)	40.1457288	0.1035502	387.693266	0.0000000
PC1	-0.3172573	0.0327265	-9.694202	0.0000000
PC2	-0.2917312	0.0419567	-6.953148	0.0000000
PC4	-0.1761277	0.0514100	-3.425939	0.0008205
PC8	0.4050280	0.0746130	5.428386	0.0000003
PC9	-0.2055781	0.0716871	-2.867715	0.0048255
PC26	0.6099109	0.1524400	4.000990	0.0001053

Describe the impacts of the predictors on Yield

Table 4 shows the fitted result of Model (4). One can see that all the predictors included in this model are significant at 0.05 significance level. To describe the impacts of them on Yield, it is necessary to first figure out the "composition" of each of the principal components.

In Appendix B, Table 11 shows the coefficients of linear combination of the original (standardized) predictors for each PC that are included in Model (4), and Figure 9 shows the proportions of variance of each PC that can be explained by the orignial predictors. Both Table 11 and Figure 9 roughly tell that PC1 is mainly accounted by the measurements of the biological starting material, PC2 is explained most by P9~P20, PC4 is especially explained by P21, PC8 is explained by P40 and P41, PC9 seems to carry both the information from biological starting material and manufacturing process, and PC26 is especially explained by P7.

Given the observation above and according to the signs of the coefficients summarized in Table 4, it thus roughly shows that after adjusting for the other variables in the model, biological starting material and P9~P21 might overall be negatively associated with Yield, and P40, P41 and P7 might be positively associated with Yield.

e. Lasso regression

```
set.seed (10)
cv.out=cv.glmnet(as.matrix(CMP)[train, 2:58],as.matrix(CMP)[train, 1], alpha=1, nfolds = 5)
plot(cv.out)
bestlam = cv.out$lambda.min
lasso.coef = predict(cv.out, type = "coefficients", s = bestlam)[,1]
abline(v = log(bestlam), col = "red")
```

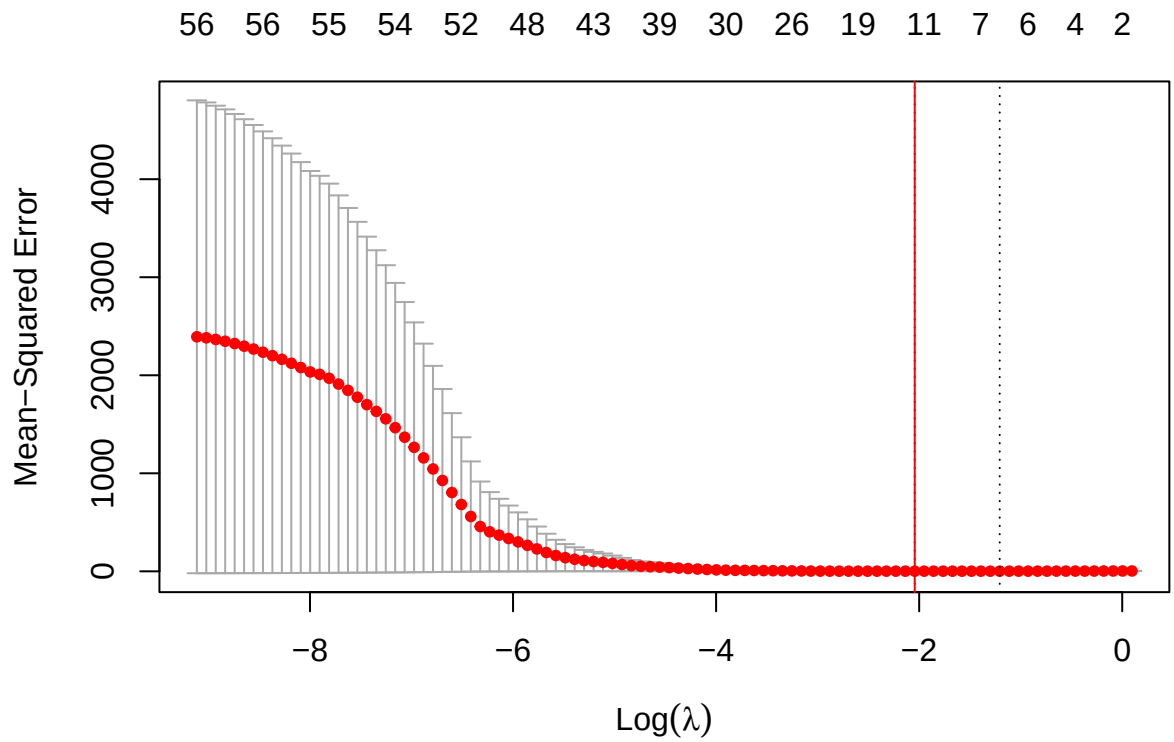


Figure 6: Cross validation with Lasso regression

Figure 6 shows how the test MSE would vary as increasing the value of $log(\lambda)$. The red points and the vertical bars respectively refer to the test MSE estimates and the corresponding confidence interval under each value of $log(\lambda)$. According to the test MSE estimates (i.e., the real points), it can be shown that $\lambda = 0.1295216$ (indicated by the red vertical line) might provide the lowest test MSE.

```
coef.names = names(lasso.coef[lasso.coef != 0])[-1]
coef.names
```

```
##  [1] "B5"  "B6"  "B7"  "P6"  "P9"  "P13" "P15" "P17" "P32" "P34" "P36" "P37"
```

According to the result of lasso regression, it follows that B5, B6, B7, P6, P9, P13, P15, P17, P32, P34, P36 and P37 might be identified as the most important predictors as they are the ones without coefficient estimates being zero. Hence, the following model is fitted using the whole training data.

$$\begin{aligned} Yield =& \beta_0 + \beta_1 B5 + \beta_2 B6 + \beta_3 B7 + \beta_4 P6 + \beta_5 P9 + \beta_6 P13 \\ &+ \beta_7 P15 + \beta_8 P17 + \beta_9 P32 + \beta_{10} P34 + \beta_{11} P36 + \beta_{12} P37 + \epsilon \end{aligned} \tag{5}$$

```
fit5 = lm(Yield~B5+B6+B7+P6+P9+P13+P15+P17+P32+P34+P36+P37, data = CMP, subset = train)
summ.kable(summary(fit5)$coef, "Summary for fitted model 5")
```

Table 5 shows the fitted result of Model (5). As one can see, some nonsiginificant predictors are present. To deal with this problem, the preditor which is the least significant in terms of p-value was removed step by

Table 5: Summary for fitted model 5

	Estimate	SE	t-value	p-value
(Intercept)	82.0640522	85.7496792	0.9570188	0.3404199
B5	0.0481598	0.0665241	0.7239451	0.4704631
B6	0.0412719	0.0387263	1.0657351	0.2886141
B7	-1.0644467	0.8263339	-1.2881558	0.2000903
P6	0.0392481	0.0424637	0.9242759	0.3571378
P9	0.3382993	0.1256319	2.6927816	0.0080660
P13	-0.4510948	0.2891869	-1.5598729	0.1213385
P15	0.0058218	0.0029452	1.9766913	0.0502958
P17	-0.0898768	0.1806409	-0.4975437	0.6196868
P32	0.1132377	0.0348087	3.2531466	0.0014707
P34	4.0793150	1.8783369	2.1717696	0.0317763
P36	-297.5865369	182.3944102	-1.6315551	0.1053104
P37	-0.6089418	0.2278367	-2.6727117	0.0085362

step until that all the predictors remaining in the model are all significant at 0.05 significance level. Notice that in each step only one predictor would be removed.

```
fit5 = update(fit5, ~.-P17)
fit5 = update(fit5, ~.-B5)
fit5 = update(fit5, ~.-B6)
fit5 = update(fit5, ~.-P6)
fit5 = update(fit5, ~.-B7)
summ.kable(summary(fit5)$coef, "Summary for fitted model 5 (revised)")
```

Table 6: Summary for fitted model 5 (revised)

	Estimate	SE	t-value	p-value
(Intercept)	-25.9902026	15.6727990	-1.658300	0.0996866
P9	0.3169448	0.1095914	2.892059	0.0044933
P13	-0.6384408	0.1726915	-3.697002	0.0003214
P15	0.0083788	0.0021454	3.905483	0.0001509
P32	0.1258384	0.0307019	4.098723	0.0000730
P34	4.4105276	1.8057649	2.442471	0.0159411
P36	-364.3076332	179.2065109	-2.032893	0.0441153
P37	-0.5570883	0.2257813	-2.467380	0.0149212

Identify the important input variables

After adopting the procedure above, Table 6 shows that P9, P13, P15, P32, P34, P36 and P37 are identified as the most important predictors.

Describe the impacts of the predictors on Yield

Table 6 shows the final fitted result of the reduced version of Model (5). One can see all the predictors are significant at 0.05 significance level. According to the sign of the coefficients, it might be able to conclude that after adjusting for the other predictors, P13, P36 and P37 might be negatively associated with Yield, and P9, P15, P32 and P34 might be positively associated with Yield.

f. Ridge regression

```
set.seed (10)
cv.out=cv.glmnet(as.matrix(CMP)[train, 2:58],as.matrix(CMP)[train, 1], alpha=0, nfolds = 5)
plot(cv.out)
bestlam = cv.out$lambda.min
ridge.coef = predict(cv.out, type = "coefficients", s = bestlam)[,1]
abline(v = log(bestlam), col = "red")
```

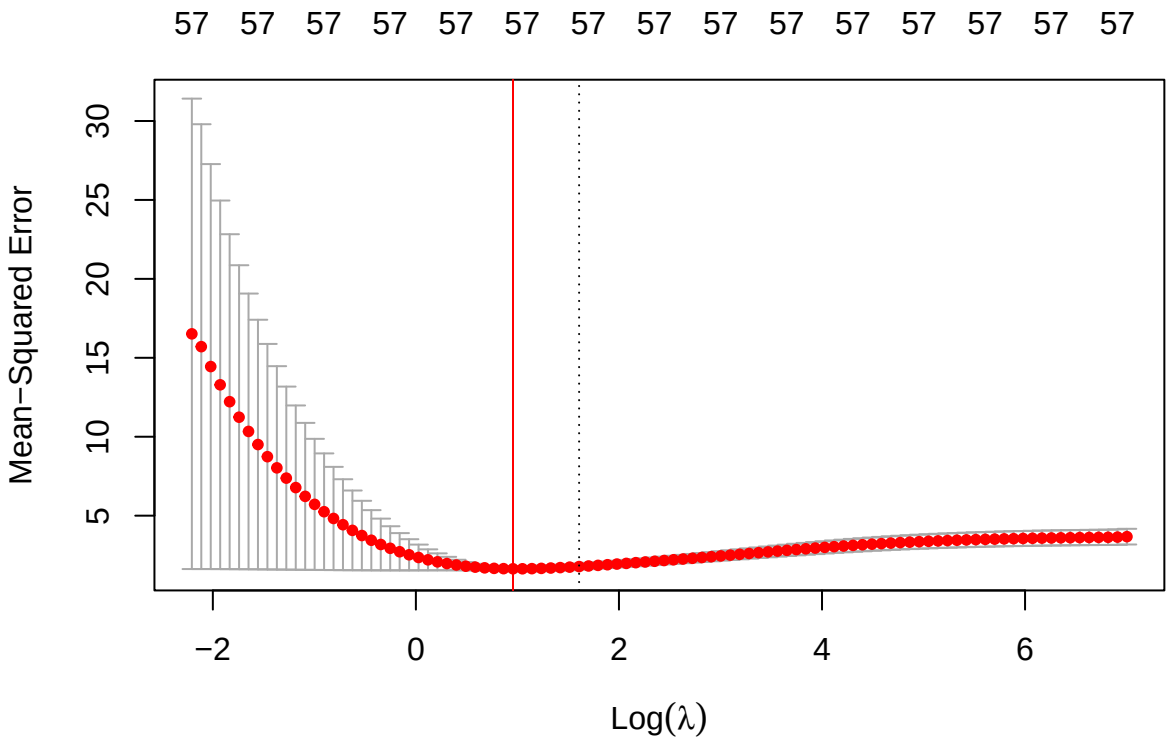
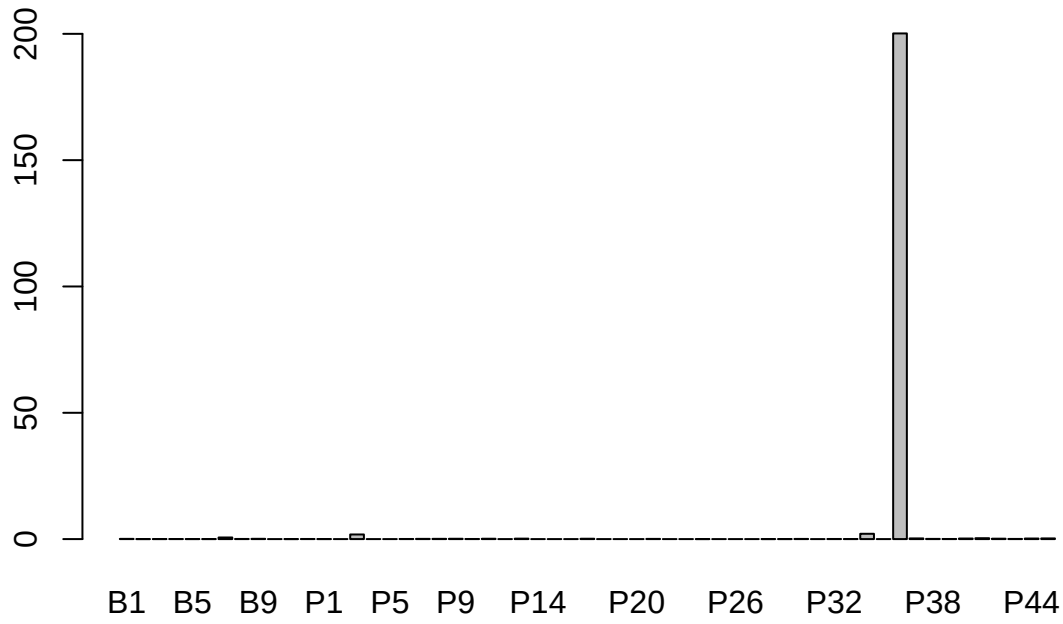


Figure 7: Cross validation with Lasso regression

Figure 7 shows how the test MSE would vary as increasing the value of $log(\lambda)$. The red points and the vertical bars respectively refer to the test MSE estimates and the corresponding confidence interval under each value of $log(\lambda)$. According to the test MSE estimates (i.e., the real points), it can be shown that $\lambda = 2.6023902$ (indicated by the red vertical line) might provide the lowest test MSE.

```
barplot(abs(ridge.coef[ridge.coef != 0][-1]))
```

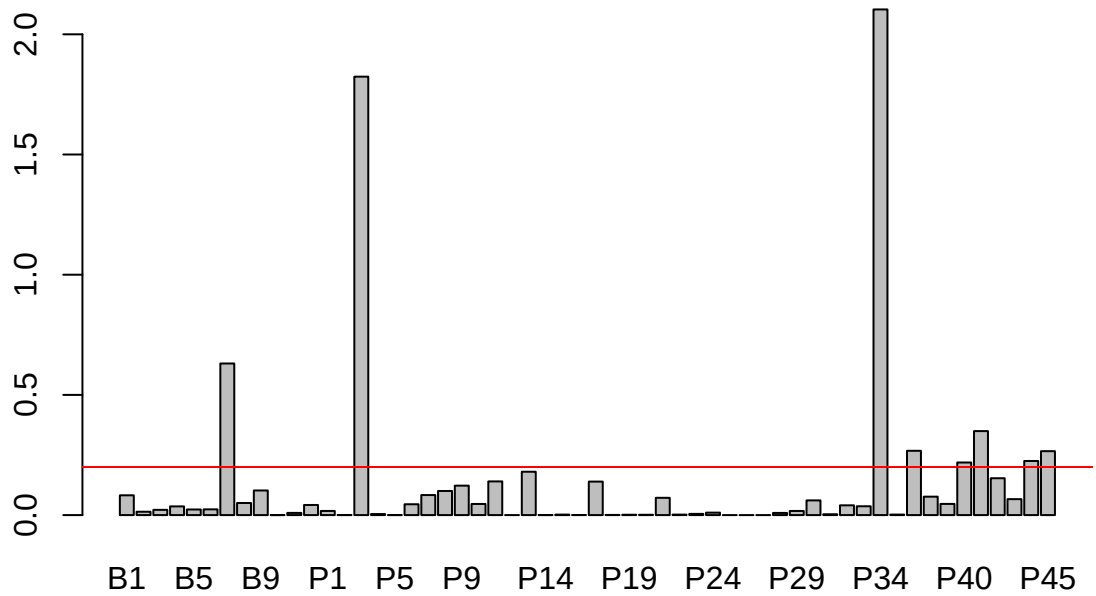



```
names(ridge.coef[ridge.coef != 0])[-1][abs(ridge.coef[-1])>5]
```

```
## [1] "P36"
```

Since the R function `glmnet` has standardized the predictors by default, we might be allowed to evaluate the importance of each predictor by their coefficient estimates in absolute value. Figure **??** shows the absolute values of the coefficients for each predictor. One can see that there is a dominated predictor P36. However, it is not reasonable to select only one predictor at this stage.

```
ridge.coef = ridge.coef[names(ridge.coef)!="P36"]
barplot(abs(ridge.coef[ridge.coef != 0][-1]))
abline(h = 0.2, col = "red")
```




```
names(ridge.coef[ridge.coef != 0])[-1][abs(ridge.coef[-1])>0.2]
```

```
## [1] "B7"  "P3"  "P34" "P37" "P40" "P41" "P44" "P45"
```

To see if there would be any potentially useful predictor except for P36, Figure **??** shows the absolute values of the coefficients for those predictors except for P36. It follows that B7, P3, P34, P37, P40, P41, P44 and P45 might also be identified as the important predictors as their coefficients have relatively large absolute values (greater than 0.25) in addition to P36. The threshold 0.25 is set here such that a few but not too many predictors would be selected at this stage. The philosophy behind such decision is essentially according to sparsity principle. By the discussion above, the following model is fitted using the whole training data.

$$\begin{aligned} Yield =& \beta_0 + \beta_1 B7 + \beta_2 P3 + \beta_3 P34 + \beta_4 P36 + \beta_5 P37 + \beta_6 P40 \\ &+ \beta_7 P41 + \beta_8 P44 + \beta_9 P45 + \epsilon \end{aligned} \tag{6}$$

```
fit6 = lm(Yield~B7+P3+P34+P36+P37+P40+P41+P44+P45, data = CMP, subset = train)
summ.kable(summary(fit6)$coef, "Summary for fitted model 6")
```

Table 7 shows the fitted result of Model (6). As one can see, only P36 is significant at 0.05 significance level. To deal with this problem, the preditor which is the least significant in terms of p-value was removed step by step until that all the predictors remaining in the model are all significant at 0.05 significance level. Notice that in each step only one predictor would be removed.

Table 7: Summary for fitted model 6

	Estimate	SE	t-value	p-value
(Intercept)	139.9917747	124.3689810	1.1256165	0.2624506
B7	-0.9250973	1.2069745	-0.7664597	0.4448252
P3	-2.3677772	6.7637461	-0.3500689	0.7268669
P34	5.2955888	2.7935716	1.8956338	0.0602809
P36	-1089.1098513	173.4097093	-6.2805587	0.0000000
P37	-0.6318902	0.3257573	-1.9397575	0.0546261
P40	-3.2802768	9.4256175	-0.3480172	0.7284035
P41	2.7769081	6.8765011	0.4038257	0.6870200
P44	1.7115928	1.6877055	1.0141537	0.3124376
P45	0.8614914	0.8123980	1.0604303	0.2909614

```
fit6 = update(fit6, ~.-P40)
fit6 = update(fit6, ~.-P41)
fit6 = update(fit6, ~.-P3)
fit6 = update(fit6, ~.-B7)
fit6 = update(fit6, ~.-P44)
fit6 = update(fit6, ~.-P45)
summ.kable(summary(fit6)$coef, "Summary for fitted model 6 (revised)")
```

Table 8: Summary for fitted model 6 (revised)

	Estimate	SE	t-value	p-value
(Intercept)	49.8248251	7.423023	6.712201	0.0000000
P34	5.3214541	2.654686	2.004551	0.0470415
P36	-1140.0548924	166.661826	-6.840528	0.0000000
P37	-0.6116958	0.318999	-1.917547	0.0573124

Identify the important input variables

After adopting the procedure above, Table 8 shows that P34, P36 and P37 are identified as the most important predictors.

Describe the impacts of the predictors on Yield

Table 8 shows the final fitted result of the reduced version of Model (6). One can see all the predictors are significant at 0.05 significance level except for P37 which is "almost" significant. According to the sign of the coefficients, it might be able to conclude that after adjusting for the other predictors, P36 and P37 might be negatively associated with Yield, and P34 might be positively associated with Yield.

0.1.6 Evaluate the prediction performance via 5-fold cross validation

```
cv.summary = matrix(NA, nrow = 3, ncol = 6)
colnames(cv.summary) = c(paste("Model", 1:4), "Model 5 (revised)", "Model 6 (revised)")
rownames(cv.summary) = c("Min.MSE", "Median.MSE", "Max.MSE")
for(i in 1:6){
  cv.errors = rep(NA, k)
```

```
  for(j in 1:k){
    if(i == 1) fit = lm(Yield~P2+P9+P17+P33+P34, data = CMP, subset = train & folds != j)
    else if(i == 2) fit = lm(Yield~P13+P15+P33, data = CMP, subset = train & folds != j)
    else if(i == 3) fit = lm(Yield~P3+P13+P15+P33+P34, data = CMP, subset = train & folds != j)
    else if(i == 4) fit = lm(Yield~PC1+PC2+PC4+PC8+PC9+PC26, data = CMP2, subset = train & folds != j)
    else if(i == 5) fit = lm(Yield~P9+P13+P15+P32+P34+P36+P37, data = CMP, subset = train & folds != j)
    else if(i == 6) fit = lm(Yield~P34+P36+P37, data = CMP, subset = train & folds != j)
    if(i != 4){
      pred = predict(fit, CMP[train & folds == j,])
      cv.errors[j] = mean((CMP[train & folds == j, 1]-pred)^2)
    }else if(i == 4){
      pred = predict(fit, CMP2[train & folds == j,])
      cv.errors[j] = mean((CMP[train & folds == j, 1]-pred)^2)
    }
  }
  cv.summary[,i] = c(min(cv.errors),
                     median(cv.errors),
                     max(cv.errors))
}
```

```
kable(as.data.frame(cv.summary), booktabs = TRUE, linesep = "", longtable = TRUE,
      align = "c", caption = "Summary of 5-fold CV for each model") %>%
  kable_styling(latex_options = c("striped", "hold_position"))
```

Table 9: Summary of 5-fold CV for each model

	Model 1	Model 2	Model 3	Model 4	Model 5 (revised)	Model 6 (revised)
Min.MSE	1.333521	1.207976	1.367067	1.145699	0.930640	2.006758
Median.MSE	1.451628	1.798037	1.423884	1.374408	1.169338	2.400548
Max.MSE	2.229987	2.303452	2.323424	2.207826	1.521460	4.168188

Table 9 shows the min, median and max of test MSE that calculated during cross validation. One can see that the revised Model (5) outperformed all the others not only in terms of the smallest median of test MSE but also in terms of the smallest range of test MSE.The test MSE for the revised Model (5) thus might be believed to be stably small. Hence, I conclude that the revised Model 5 is the best model that I have found so far at lest in terms of prediction performance.

0.1.7 Evaluate the prediction performance via testing error on testing set

```
test.summary = matrix(NA, nrow = 1, ncol = 6)
colnames(test.summary) = c(paste("Model", 1:4), "Model 5 (revised)", "Model 6 (revised)")
rownames(test.summary) = c("RMSE")
par(mfrow = c(2, 3), mar = c(5, 4.5, 4, 2) + 0.1)
for(i in 1:6){
  if(i == 1) fit = lm(Yield~P2+P9+P17+P33+P34, data = CMP, subset = train)
  else if(i == 2) fit = lm(Yield~P13+P15+P33, data = CMP, subset = train)
  else if(i == 3) fit = lm(Yield~P3+P13+P15+P33+P34, data = CMP, subset = train)
  else if(i == 4) fit = lm(Yield~PC1+PC2+PC4+PC8+PC9+PC26, data = CMP2, subset = train)
  else if(i == 5) fit = lm(Yield~P9+P13+P15+P32+P34+P36+P37, data = CMP, subset = train)
```

```
  else if(i == 6) fit = lm(Yield~P34+P36+P37, data = CMP, subset = train)
  if(i != 4){
    pred = as.data.frame(predict(fit, CMP[test,], interval = "prediction"))
    test.summary[,i] = sqrt(mean((CMP[test, 1]-pred$fit)^2))
    plotCI(CMP[test, 1], pred$fit, ui = pred$upr, li = pred$lwr,
           xlab = expression(Yield), ylab = expression(hat(Yield)),
           main = paste("Model", i))
  }else if(i == 4){
    pred = as.data.frame(predict(fit, CMP2[test,], interval = "prediction"))
    test.summary[,i] = sqrt(mean((CMP2[test, 1]-pred$fit)^2))
    plotCI(CMP2[test, 1], pred$fit, ui = pred$upr, li = pred$lwr,
           xlab = expression(Yield), ylab = expression(hat(Yield)),
           main = paste("Model", i))
  }
  abline(a = 0, b = 1, col = "red")
}
```

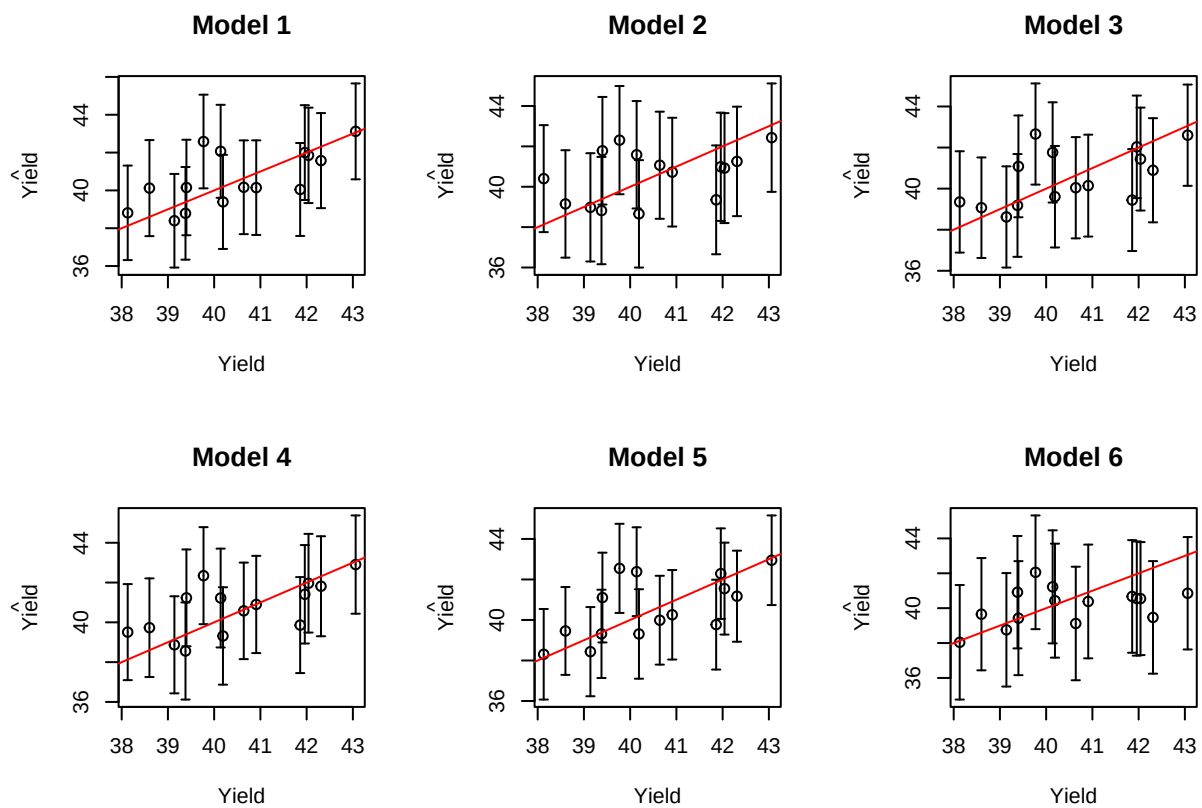


Figure 8: Relation between predicted Yield and true Yield

```
kable(as.data.frame(test.summary), booktabs = TRUE, linesep = "",
      align = "c", caption = "Summary of testing error for each model") %>%
  kable_styling(latex_options = c("hold_position"))
```

Figure 8 shows the relation between predicted value of Yield and the true observed value of Yield from which

Table 10: Summary of testing error for each model

	Model 1	Model 2	Model 3	Model 4	Model 5 (revised)	Model 6 (revised)
RMSE	1.188796	1.473468	1.304892	1.166762	1.279897	1.442108

one might think that there seems to be no big difference among the six models in terms of prediction performance. Nevertheless, Table 10 shows the test root mean squared error (RMSE) of each model, suggesting that Model 4 might be the one with the lowest RMSE. From this viewpoint, one might conclude that Model 4 could be the best model in terms of test RMSE (or test MSE) on the testing data set. Notice that such conclusion would seem to contradict with that made according to CV test MSE where revised Model 5 was said to be the best model, and we have no more information to further compare the two. Hence, to be conservative, we conclude that Model 4 and revised Model 5 are comparable in prediction performance, and thus both could be regarded as the best models that might outperform the rest.

0.1.8 Brief analysis summary

Though the indices of prediction performance summarized in Table 9 and Table 10 actually do not differ really much among the six models, we simply conclude that Model 4 and revised Model 5 could be the best models since they seem to have the lowest test MSE. However, it might not be appropriate to say the other models are not good and discard them directly. Those with a bit higher test MSE might actually enjoy other advantages such as higher interpretability. It would be more suitable to evaluate the value of application of them after more discussion with some related experts.

As for the important variables in predicting Yield, it is found that P33 ane P34 might be regarded as the most important ones since they are selected for the most times (P33 is selected in 3 models and P34 is selected in 4 models). Furthermore, one might also be allowed to claim that P33 and P34 might both be positively associated with Yield in some degree of confidence since the coefficients of them are consistently positive in those models that have selected the two predictors.

0.2 Problem 2: Data with Outliers

For iid data $X_i, i = 1, 2, ..., n$ from some distribution, the sample standard deviation $\hat{\sigma}$ is often used to estimate the population deviation σ:

$$\hat{\sigma} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2}.$$

But sometimes, the observed data are contaminated with extreme values or outliers, due to recording errors or other reasons. In order to obtain a robust estimate for σ, $\tilde{\sigma}$ is considered:

$$\tilde{\sigma} = 1.4826 \cdot med_{1 \le i \le n}\{|X_i - X_{med}|\},$$

where X_{med} denotes the sample median of $\{X_i : 1 \le i \le n\}$.

Apply the nonparametric bootstrap method to evaluate the sampling distributions of $\tilde{\sigma}$ and $\hat{\sigma}$, and their estimation precision $var(\tilde{\sigma})$ and $var(\hat{\sigma})$.

The data for problem 2: **hw3_problem2.csv**

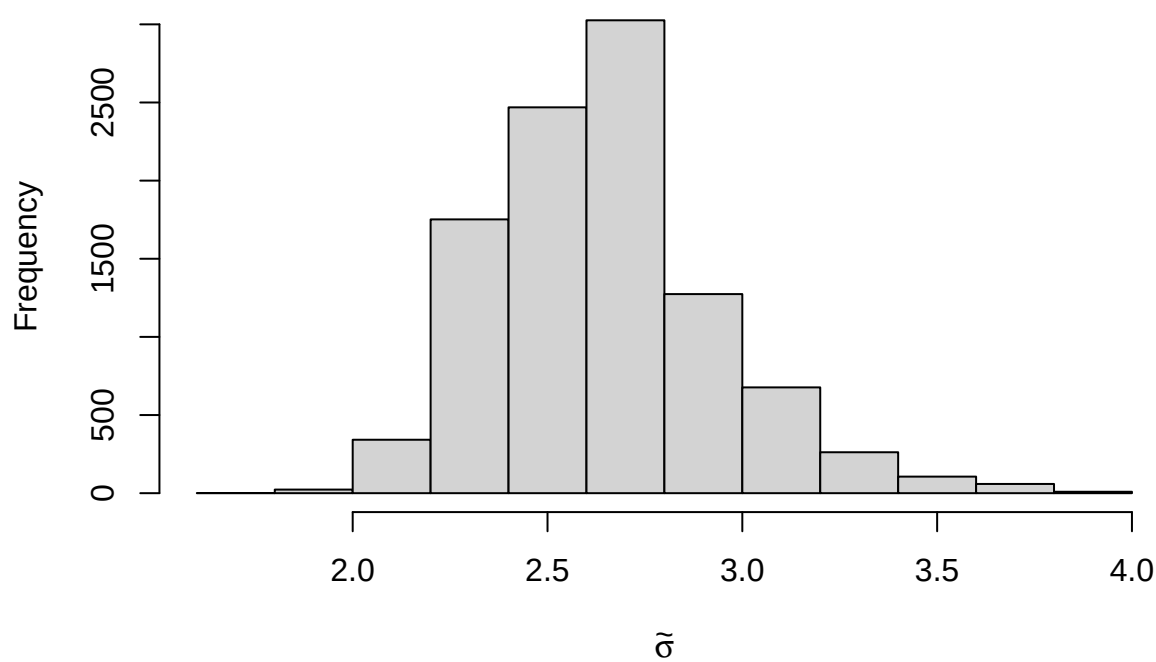
0.2.1 Bootstrap for the sampling distribution of $\tilde{\sigma}$

Read data:

```
dat = read.csv("hw3_problem2.csv")
head(dat)
```

```
##   ID         x
## 1  1  7.669894
## 2  2 10.805711
## 3  3  6.913241
## 4  4 10.366771
## 5  5 14.669816
## 6  6 11.816753
```

```
B = 10000
B.res = rep(NA, B)
set.seed(10)
for(b in 1:B){
  sam = sample(dat$x, replace = TRUE)
  B.res[b] = 1.4826*median(abs(sam - median(sam)))
}
hist(B.res, xlab = expression(tilde(sigma)), main = "")
```



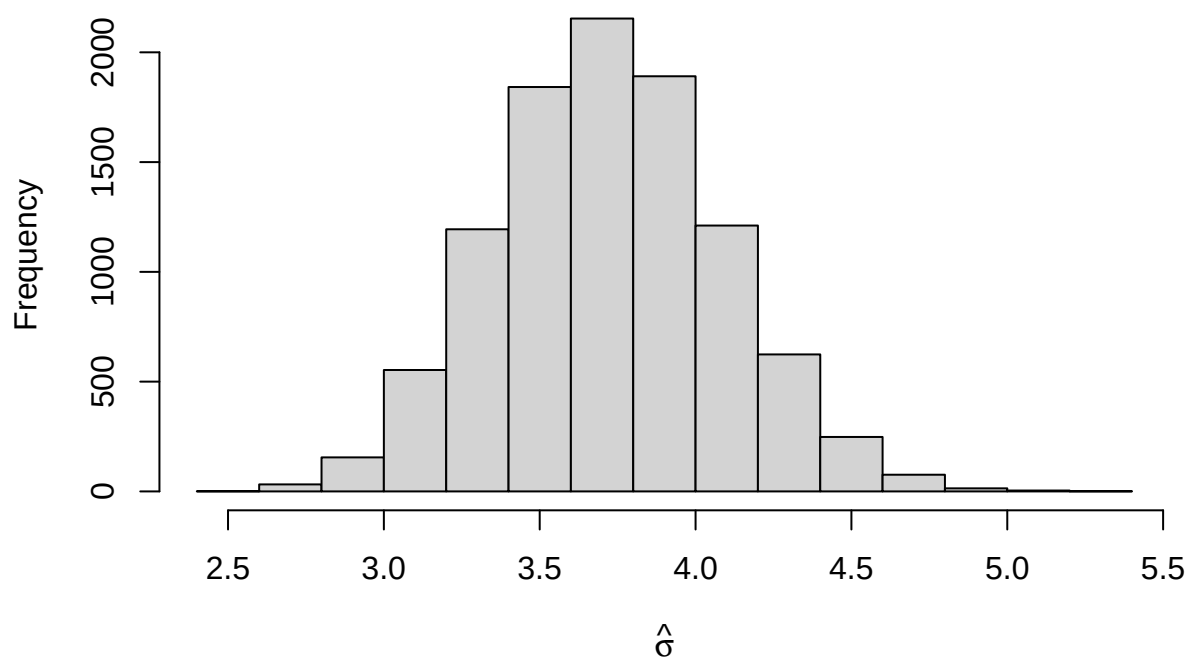
```
var(B.res)
```

```
## [1] 0.084577
```

The estimated $var(\tilde{\sigma})$ is 0.084577.

0.2.2 Bootstrap for the sampling distribution of $\hat{\sigma}$

```
B = 10000
B.res = rep(NA, B)
set.seed(10)
for(b in 1:B){
  sam = sample(dat$x, replace = TRUE)
  B.res[b] = sd(sam)
}
hist(B.res, xlab = expression(hat(sigma)), main = "")
```




```
var(B.res)
```

```
## [1] 0.1315167
```

The estimated $var(\hat{\sigma})$ is 0.1315167.

0.3 Appendix

0.3.1 A. Summary for the variables in Problem 1.

```
stat = round(sapply(CMP, summary),2)
kable(as.data.frame(stat[,1:10]), format = "latex", booktabs = TRUE,align = "c",
    escape = FALSE,linesep = "") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

	Yield	B1	B2	B3	B4	B5	B6	B7	B8	B9
Min.	35.25	4.58	46.87	57.56	9.38	13.24	40.60	100.00	15.88	11.73
1st Qu.	38.69	5.90	52.67	64.34	11.15	17.09	45.84	100.00	17.00	12.62
Median	39.91	6.28	54.50	67.04	12.02	18.26	47.89	100.00	17.41	12.88
Mean	40.13	6.31	55.32	67.79	12.16	18.42	48.60	100.02	17.39	12.87
3rd Qu.	41.45	6.76	58.18	71.05	12.73	19.28	50.85	100.00	17.76	13.14
Max.	46.34	8.81	64.75	78.25	23.09	24.85	59.38	100.83	18.85	14.08

```
kable(as.data.frame(stat[,11:20]), format = "latex", booktabs = TRUE,align = "c",
    escape = FALSE,linesep = "") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

	B10	B11	B12	P1	P2	P3	P4	P5	P6	P7
Min.	1.77	135.81	18.35	8.70	0.00	1.47	918.00	923.00	203.00	177.00
1st Qu.	2.45	143.65	19.71	10.80	19.50	1.53	927.75	984.27	205.70	177.00
Median	2.62	145.75	20.03	11.40	21.10	1.54	934.00	996.45	206.60	177.00
Mean	2.73	146.57	20.12	11.35	18.01	1.54	932.26	996.69	207.25	177.49
3rd Qu.	2.91	149.03	20.50	12.10	21.50	1.55	937.00	1005.15	208.70	178.00
Max.	6.87	158.73	22.21	14.10	22.50	1.60	946.00	1171.20	227.40	178.00

```
kable(as.data.frame(stat[,21:30]), format = "latex", booktabs = TRUE,align = "c",
    escape = FALSE,linesep = "") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

	P8	P9	P10	P11	P12	P13	P14	P15	P16	P17
Min.	177.00	38.89	7.50	7.50	0.00	32.10	4701.00	5904.00	4441.00	31.30
1st Qu.	177.00	44.77	8.70	8.97	0.00	33.90	4828.00	6008.00	4559.00	33.50
Median	178.00	45.73	9.00	9.30	0.00	34.60	4854.00	6030.00	4583.00	34.30
Mean	177.57	45.62	9.16	9.34	987.61	34.51	4850.65	6030.99	4584.05	34.31
3rd Qu.	178.00	46.48	9.50	9.80	0.00	35.12	4880.50	6056.25	4614.00	35.00
Max.	178.00	49.36	11.60	11.50	4549.00	38.60	5055.00	6213.00	4852.00	40.00

```
kable(as.data.frame(stat[,31:40]), format = "latex", booktabs = TRUE,align = "c",
    escape = FALSE,linesep = "") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

	P18	P19	P20	P21	P22	P23	P24	P25	P26	P27
Min.	4612.00	5890.00	4392.00	-1.8	1.00	1.00	1.00	0.00	0.00	0.00
1st Qu.	4813.00	5997.00	4551.75	-0.6	3.00	2.00	4.75	4829.00	6018.25	4555.50
Median	4834.00	6018.50	4579.00	-0.3	5.00	3.00	8.00	4853.50	6041.50	4584.50
Mean	4835.78	6022.63	4576.50	-0.2	5.55	3.19	9.22	4822.76	6005.72	4553.46
3rd Qu.	4857.25	6039.00	4601.00	0.0	8.00	4.00	14.00	4876.25	6061.00	4606.00
Max.	4971.00	6146.00	4759.00	3.0	12.00	6.00	23.00	4990.00	6161.00	4710.00

```
kable(as.data.frame(stat[,41:50]), format = "latex", booktabs = TRUE,align = "c",
    escape = FALSE,linesep = "") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

	P28	P29	P30	P31	P32	P33	P34	P35	P36	P37
Min.	0.00	0.00	0.00	0.00	143.00	56.00	2.30	463.00	0.02	0.00
1st Qu.	0.00	19.70	8.80	70.30	155.00	62.00	2.50	490.00	0.02	0.70
Median	10.30	19.90	9.10	70.90	158.00	63.00	2.50	496.00	0.02	1.00
Mean	6.18	19.89	9.11	70.27	157.99	63.31	2.49	496.01	0.02	0.99
3rd Qu.	10.70	20.30	9.70	71.40	161.00	65.00	2.50	502.00	0.02	1.22
Max.	11.50	22.00	11.20	72.50	170.00	69.00	2.60	522.00	0.02	2.00

```
kable(as.data.frame(stat[,51:58]), format = "latex", booktabs = TRUE,align = "c",
      escape = FALSE,linesep = "") %>%
    kable_styling(latex_options = c("striped", "hold_position"))
```

	P38	P39	P40	P41	P42	P43	P44	P45
Min.	2.00	0.00	0.00	0.00	10.50	0.00	1.60	1.8
1st Qu.	2.00	7.10	0.00	0.00	11.40	0.50	1.80	2.1
Median	3.00	7.20	0.00	0.00	11.60	0.80	1.90	2.2
Mean	2.63	7.02	0.02	0.03	11.54	0.89	1.86	2.2
3rd Qu.	3.00	7.30	0.00	0.00	11.70	1.00	1.90	2.3
Max.	3.00	7.50	0.10	0.20	12.10	11.00	2.10	2.6

0.3.2 B. Composition of each principal component in Model (4)

1. Summary for the coefficients.

```
loadings = PC[, c(1, 2, 4, 8, 9, 26)]
rownames(loadings) = colnames(CMP)[-1]
colnames(loadings) = paste("PC", c(1, 2, 4, 8, 9, 26), sep = "")
kable(as.data.frame(loadings), booktabs = TRUE, linesep = "", longtable = TRUE,
      caption = "Linear combination for the PC in Model 4") %>%
  kable_styling(latex_options = c("striped", "hold_position"))
```

Table 11: Linear combination for the PC in Model 4

	PC1	PC2	PC4	PC8	PC9	PC26
B1	-0.2628164	0.0340750	-0.0077229	0.0215272	-0.0046946	0.0165070
B2	-0.2932181	0.0057882	-0.0716302	-0.0276624	0.0237114	0.0567134
B3	-0.2732496	-0.0180662	-0.0964833	-0.0613165	0.0329182	0.0896996
B4	-0.2487303	0.0210387	-0.0440406	-0.0137206	-0.0779470	0.0726793
B5	-0.0933751	0.0728004	0.0012307	0.0047713	0.0369391	0.0370090
B6	-0.2787476	0.0008509	-0.0604762	-0.0506404	0.0585378	0.0994768
B7	-0.0096105	-0.0627652	0.1583589	-0.1133663	0.2931806	0.1388718
B8	-0.2583365	-0.0460015	-0.0453311	-0.0792060	0.1480162	0.0022594
B9	-0.1547505	-0.0411532	-0.1087974	-0.1693462	0.1553147	-0.1012846
B10	-0.1923847	0.0431545	-0.0281589	-0.0398101	-0.0463018	-0.0524297
B11	-0.2539513	-0.0280001	-0.0992015	-0.1655466	0.2192741	0.0414992
B12	-0.2441513	-0.0530788	-0.0616849	-0.1710845	0.2425177	0.0245416
P1	0.1018305	-0.0120398	-0.2373072	0.0801292	-0.0156542	-0.0854769
P2	0.1631017	-0.1025248	-0.0532337	0.0747996	0.0447326	0.0140422
P3	0.0357025	0.0720546	-0.1831704	-0.1148034	-0.0969936	-0.1291240
P4	0.1615035	0.0744679	-0.2062758	-0.0293069	-0.0526361	0.1867409
P5	-0.0160872	-0.0642788	-0.1548069	-0.0739726	-0.0319893	-0.2526822
P6	-0.0745542	-0.1467085	-0.0564518	0.1648467	0.0356304	-0.1367506
P7	0.0192260	-0.0144122	0.0304537	-0.0914657	-0.0575208	-0.4658968
P8	0.0152733	-0.0106937	0.0382896	-0.0246307	-0.0514224	0.0358733
P9	-0.0710495	-0.3041400	-0.1149203	0.0615128	0.0507497	0.1894861
P10	-0.0209133	-0.2375059	0.2555526	-0.0310320	-0.0162901	0.0983641
P11	0.0076234	-0.2816758	-0.1227058	-0.0005513	-0.0333727	0.0079781
P12	-0.1156203	-0.2159799	0.1283593	0.0816648	-0.0210147	-0.2349194
P13	0.0507897	0.3323887	0.0028455	-0.1240159	0.0283825	-0.0794331
P14	-0.0359675	0.2480347	-0.3066536	0.0685129	0.0235230	-0.0404535
P15	-0.0804650	0.1752155	-0.3250360	0.1367596	0.0023293	0.0705703
P16	0.0401443	0.2149620	-0.2493160	0.0623066	0.0611612	-0.0957363
P17	-0.0030995	0.2784372	0.2301907	-0.1502933	0.0031137	-0.1074367
P18	-0.0718353	0.2947097	0.1603230	0.0755585	0.0393341	0.0125738
P19	-0.1335756	0.2329257	0.1332143	0.1341330	-0.0150515	0.2133921
P20	0.0210744	0.2555337	0.0774107	0.0858600	0.0399847	0.0064984
P21	-0.0771242	0.0133337	0.3951943	-0.0854574	-0.0346949	-0.0741126
P22	0.0053104	-0.0153102	-0.0507296	-0.0100653	0.3424924	-0.1230204
P23	0.0350683	-0.0062301	0.0411617	0.1115775	0.2591792	-0.0060939
P24	0.1237576	0.0204860	0.0468711	0.1250090	-0.1394103	0.1014093
P25	-0.0387844	0.1330840	0.0865203	-0.0111156	0.0373784	-0.0232115
P26	-0.0415176	0.1197913	0.0812807	-0.0041436	0.0353583	-0.0076977
P27	-0.0268498	0.1287395	0.0809476	-0.0077675	0.0452429	-0.0225944
P28	-0.2135496	-0.0221069	0.1873349	-0.0037962	-0.0766831	-0.3322735
P29	-0.0842810	0.1203729	0.0961448	0.0269268	0.0243135	0.0146026
P30	-0.0104676	-0.1064924	-0.0214756	-0.0094865	0.0105275	-0.0013463
P31	-0.0109949	0.1307326	0.0737260	-0.0391228	0.0468111	-0.0256811
P32	-0.2298042	0.0250085	0.0300105	0.2358644	-0.2133874	0.1072535
P33	-0.2045966	0.0635886	-0.0152873	0.1515955	-0.2957153	0.0945579
P34	0.0209844	-0.0828212	0.1035406	0.1320862	0.1476481	0.0694660
P35	0.0750045	0.0178719	0.0568047	-0.0735573	0.0652453	0.3036939
P36	0.2066204	-0.0187645	0.0225177	-0.2388173	0.1850860	0.1183286
P37	0.0264375	0.0222284	-0.0713748	-0.0292377	0.1065416	-0.2508060
P38	0.0207507	0.0126903	0.0034304	-0.0072369	0.1557098	-0.0513890
P39	0.0395943	-0.0375965	0.0883549	0.0152177	-0.2221961	-0.0357217
P40	0.0063987	0.0098024	-0.0550863	-0.4868441	-0.2609949	0.1050497

P41	-0.0091063	0.0098612	-0.0618483	-0.4792374	-0.2426646	0.1140370
P42	0.1032316	0.0591008	0.0427739	-0.1116320	0.1489256	0.0795584
P43	-0.0633804	-0.0677229	0.1157054	-0.0119802	-0.0557710	0.0865568
P44	-0.0671056	-0.0918186	-0.0537833	0.0513680	-0.2179472	-0.0420216
P45	-0.0140502	-0.0228096	-0.0194106	-0.1764867	-0.1193560	-0.1259471

2. Proportions of variance of each PC accounted by the original (standardized) predictors

```
par(mfrow = c(2, 3))
for(i in 1:6){
  barplot(loadings[,i]^2, ylab = "coefficients", main = colnames(loadings)[i])
}
```

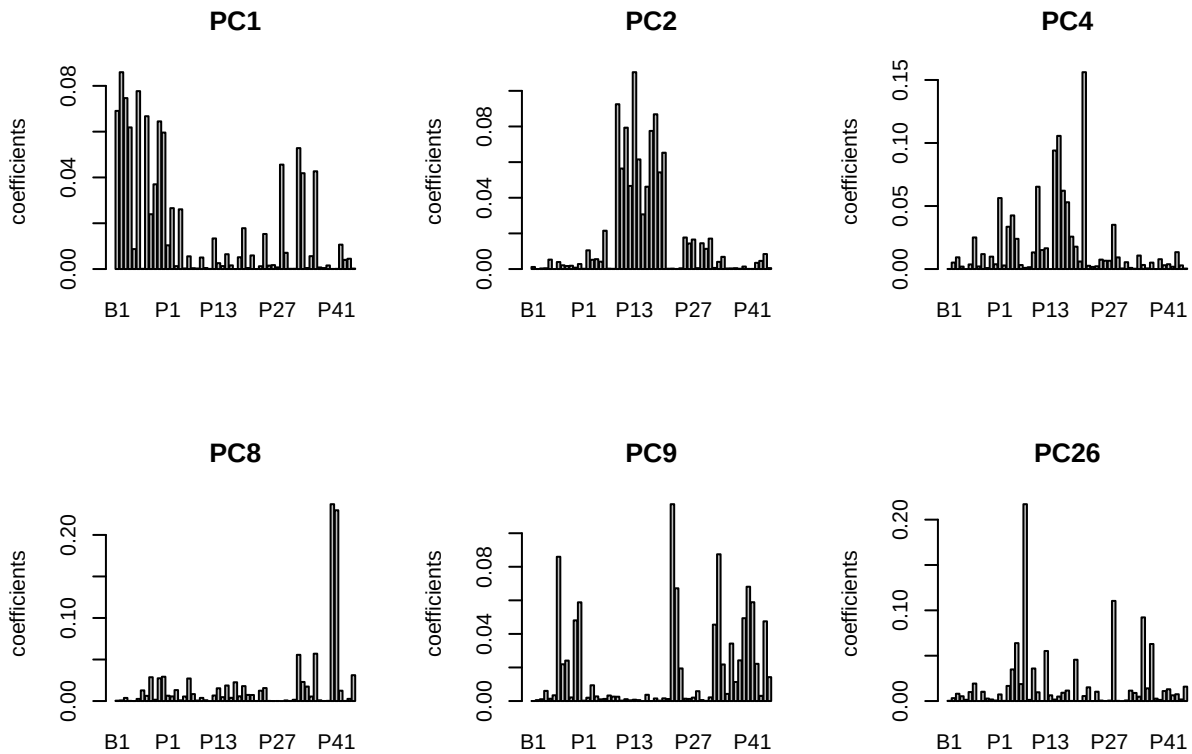


Figure 9: Proportion of variance accounted by the original predictors